

FICHA TÉCNICA

LexSign

Aplicación Multiplataforma de Gestión Jurídica con Firma Electrónica

Fecha de Generación: 22 de May de 2026

1. INFORMACIÓN GENERAL

Nombre del Proyecto	LexSign
Tipo de Aplicación	Aplicación Multiplataforma Móvil
Plataformas Soportadas	Android, iOS
Versión Actual	1.0.0
Estado del Proyecto	En Desarrollo
Autor/Equipo	MiguelBonilla-sys

2. DESCRIPCIÓN DEL PROYECTO

LexSign es una plataforma integral de gestión jurídica y consultoría legal que permite a clientes conectar con abogados profesionales, gestionar consultas, y firmar documentos electrónicamente de forma segura. La aplicación integra tecnología de firma digital avanzada (CAMERFIRMA) con una base de datos robusta en la nube (Supabase) para garantizar la integridad, seguridad y trazabilidad de todos los documentos legales.

3. STACK TECNOLÓGICO

Categoría	Tecnología
Lenguaje Principal	Kotlin
Framework UI	Jetpack Compose Multiplatform
Backend	Supabase (PostgreSQL)
Autenticación	Supabase Auth
Almacenamiento	Supabase Storage
Firma Electrónica	CAMERFIRMA API
Build System	Gradle + Kotlin DSL

4. CARACTERÍSTICAS PRINCIPALES

- **Gestión de Consultas Jurídicas:** Permite a clientes crear consultas y conectar con abogados especializados
- **Gestoría de Documentos:** Carga, organización y almacenamiento seguro de documentos legales
- **Firma Electrónica Avanzada:** Integración con CAMERFIRMA para firmas digitales válidas legalmente

- **Autenticación Segura:** Sistema de login con hash PBKDF2-HMAC-SHA256
- **Row Level Security (RLS):** Control granular de acceso a datos basado en rol del usuario
- **Auditoría Completa:** Registro automático de todas las operaciones (audit_logs)
- **Notificaciones en Tiempo Real:** Sistema de notificaciones para eventos importantes
- **Interfaz Multiplataforma:** Una sola base de código para Android e iOS

5. ARQUITECTURA DEL SISTEMA

Patrón Arquitectónico: MVVM (Model-View-ViewModel) con capas bien definidas.

Capas de la Aplicación:

- **UI Layer:** Screens Compose, ViewModels, Navigation graph
- **Data Layer:** Repositories, Models, Remote API integration
- **Session/Storage Layer:** SessionManager, Supabase client, caching

Flujo de Datos: Acciones de usuario → ViewModel → Repository → Supabase → Response → Update UI State → Recompose

Base de Datos: PostgreSQL mediante Supabase con tablas principales: users, consultas, documentos, notificaciones, audit_logs

6. MEDIDAS DE SEGURIDAD

- **Hash de Contraseñas:** PBKDF2-HMAC-SHA256 con 10,000 iteraciones y salt aleatorio de 16 bytes
- **Row Level Security (RLS):** Todas las tablas tienen RLS habilitado para control de acceso granular
- **Autenticación:** Token-based con Supabase Auth
- **Auditoría:** Triggers automáticos registran cambios con datos anteriores y nuevos
- **Integridad de Documentos:** Sello de tiempo verificado en firmas electrónicas
- **Encriptación:** Conexiones HTTPS/TLS para toda comunicación con backend

7. REQUISITOS DEL SISTEMA

Requisito	Especificación
Versión Kotlin	Compatible con versiones recientes
Android Mínimo	API 21+
iOS Mínimo	iOS 12.0+
Java Runtime	JDK 11 o superior
Gradle	8.0+
Conexión	Internet requerido

8. ESTRUCTURA DEL PROYECTO

Directorio Raíz:

- composeApp/ - Aplicación principal Compose Multiplatform
- ■ ■ ■ src/commonMain/kotlin/com/example/proyecto/
- ■ ■ ■ data/ - Modelos, repositorios, gestión de sesión
- ■ ■ ■ ui/ - Screens, ViewModels, tema Material
- ■ ■ ■ navigation/ - Gráfo de navegación
- iosApp/ - Punto de entrada iOS
- supabase/ - Configuración de Supabase

- docs/ - Documentación técnica del proyecto
- gradle/ - Configuración de build

9. FLUJO DE DESARROLLO

Compilación y Build:

Android: ./gradlew :composeApp:assembleDebug
iOS: Desde Xcode usando configuración estándar de iOS

Patrón de Desarrollo: MVVM con StateFlow para gestión de estado reactiva
Versionado: Git con rama main y ramas de feature
Documentación: Disponible en carpeta docs/ con múltiples especificaciones técnicas

10. DEPENDENCIAS CLAVE

- Supabase Client - Backend-as-a-Service con PostgreSQL
- Kotlin Coroutines - Programación asincrónica
- Jetpack Compose - Framework UI declarativo
- Material Design 3 - Sistema de diseño
- Serialización JSON - Para manejo de datos
- SecurityCrypto - Para operaciones de seguridad

11. ENDPOINTS PRINCIPALES (API REST)

Tabla	Operaciones
/users	GET, POST (CRUD de usuarios)
/consultas	GET, POST, PUT (Gestión de consultas)
/documentos	GET, POST, DELETE (Gestión de documentos)
/notificaciones	GET (Consulta de notificaciones)
/audit_logs	GET (Consulta de logs de auditoría)

12. PRÓXIMOS PASOS Y MEJORAS

- Optimización de rendimiento en sincronización de datos
- Implementación de caché local offline-first
- Mejora de interfaz UI/UX
- Integración con más proveedores de firma electrónica
- Análisis y reportes avanzados
- Testing automatizado completo (unit, integration, E2E)